

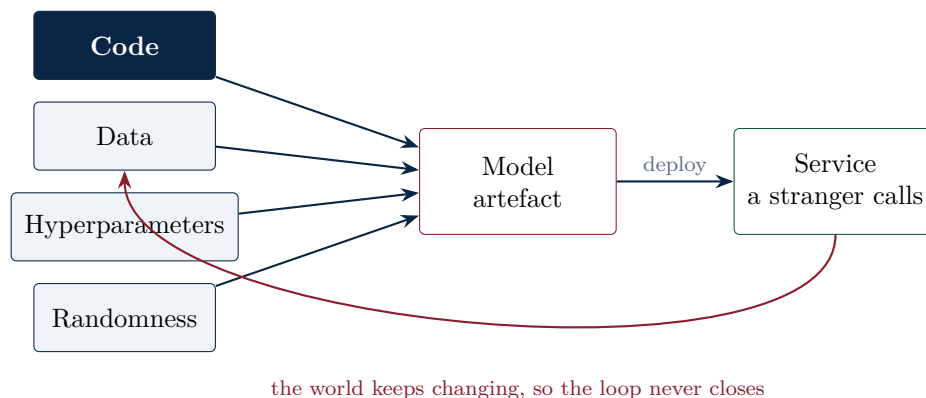
BENNETT UNIVERSITY

School of Computer Science Engineering and Technology

MLOps

A Technical Handbook

What the discipline is, why it belongs in the curriculum,
and how to open the course



Contents

I	What MLOps Actually Is	4
1	The definition worth using in the room	4
2	CACE: why you cannot unit test a model	4
3	The technical debt taxonomy	5
4	The anatomy of a production system	7
5	Training and serving skew, drawn	8
6	Lineage, formally	8
7	Reproducibility, and its limits	8
8	The three shifts, decomposed	9
9	The quiet failure	9
10	Deployment strategies and rollback semantics	10
11	The testing pyramid for machine learning	11
12	The maturity model	11
II	Why It Must Be Taught	12
13	The curriculum gap, stated without insult	12
14	The economic argument, in one line	12
15	The role map	12
16	What interviews actually ask, once they move past data structures	13
17	The portfolio asymmetry	13
III	The Skill Stack	14
18	What they must already own, and what they usually lack	14
19	The stack we build, layer by layer	15

20 What compounds beyond this course	15
IV Opening the Course	16
21 The cold open, twelve minutes	16
22 The four lines for the board	16
23 Hard questions from sharp students	17
24 The sentence to end the first lecture on	17
V Reference	18
25 Vocabulary to use fluently from day one	18
26 Primary sources	19

List of Figures

1	The one distinction the whole discipline rests on. On the left, behaviour is fully specified by something a human wrote. On the right, three of the four inputs are outside the source tree, and one of them, the data, keeps moving after release.	4
2	The CACE property. Entanglement is not a flaw in a particular model; it is a consequence of joint optimisation, and it is why the discipline replaces line level testing with distributional and behavioural testing.	5
3	The proportion students never expect. In a real production system the modelling code is a small fraction of the total; the rest is the surrounding engineering, which is exactly the material of this course.	6
4	The four subsystems and the one arrow that matters. Without the red return path the system is a line, not a loop, and everything downstream decays silently.	7
5	Training and serving skew: the single most common production failure in machine learning, and the reason feature stores and shared transformation libraries exist. . . .	8
6	The lineage chain. In an audit, an incident review or any regulated deployment, this graph is the deliverable. Tooling exists to record it automatically, which is the entire argument for a tracking server and a registry.	8
7	Solid navy is the reference window, dashed red the current window. The engineering consequence is the asymmetry: input drift is cheap to monitor because it needs no ground truth, while concept drift is expensive because it needs labels that arrive late or never.	9
8	The asymmetry that justifies the entire monitoring module. A web server tells you when it crashes. A model does not tell you when it starts being wrong.	9
9	Four ways to put a new model in front of users, ordered by risk appetite. Shadow buys certainty at the cost of compute; canary buys early signal under real load; blue and green buys instant rollback at the cost of double infrastructure; A and B testing buys causal evidence at the cost of time.	10
10	Wide at the base, because most production incidents are data incidents rather than model incidents. Invariance tests are the machine learning equivalent of unit tests: if the customer name changes the prediction must not; if distance increases the predicted time must not decrease.	11
11	The three levels of automation. Note what changes at level two: the deployed artefact stops being a model and becomes a pipeline that produces models.	11
12	Accuracy is the tip. Everything that keeps a model trustworthy for strangers sits below the waterline, invisible until it is missing, and every box below the line is a session in this syllabus.	12
13	The single most persuasive slide you can show a fifth semester student. The right hand box converts an interview from a quiz into a conversation about their own work. . . .	13
14	The semester as a ladder. Nothing is thrown away, and every rung produces an artefact a student can show to somebody who is deciding whether to hire them.	15
15	The opening sequence. Three physical demonstrations before a single slide of syllabus. Each takes under three minutes and each lands an idea the rest of the semester rests on.	16

PART I

What MLOps Actually Is

1 The definition worth using in the room

Traditional software is a function of code alone. Given the same code and the same input you get the same output, and every failure is traceable to a line somebody wrote.

A machine learning system is a function of **code, data, hyperparameters and randomness**, and its behaviour is only partly specified by the code. The weights are compiled by an optimiser from a dataset that nobody wrote and nobody fully inspected. This is the whole difference, and everything else in this handbook follows from it.

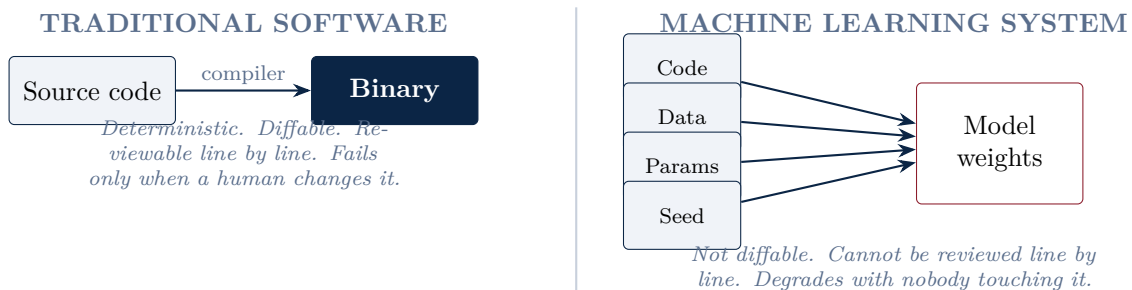


Figure 1: The one distinction the whole discipline rests on. On the left, behaviour is fully specified by something a human wrote. On the right, three of the four inputs are outside the source tree, and one of them, the data, keeps moving after release.

The sentence for the board

In software you version code. In machine learning you must version code, data and model, because any one of the three changing changes the behaviour of the system, and only one of the three is under your direct control.

2 CACE: why you cannot unit test a model

Sculley and colleagues at Google named the underlying property in 2015: **Changing Anything Changes Everything**. Machine learning erodes abstraction boundaries. In well designed software you reason about a module in isolation because its interface constrains it. In a model, every input feature is entangled with every other through the learned weights. Remove one feature, or shift its distribution, and the learned importance of every remaining feature changes.

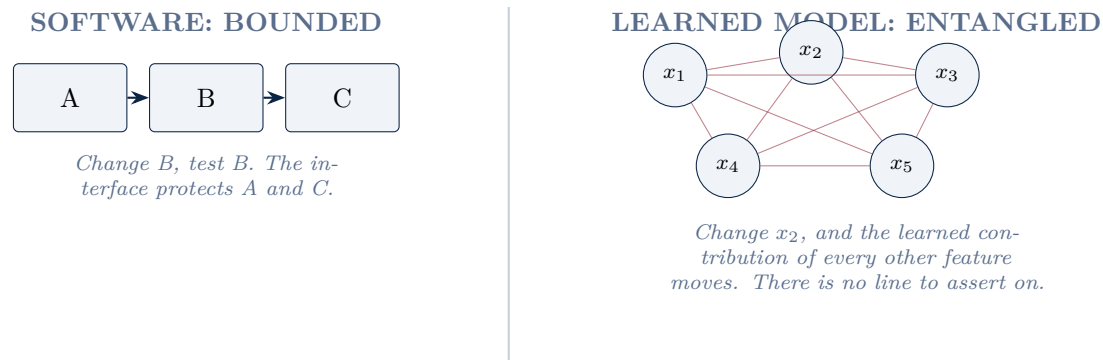


Figure 2: The CACE property. Entanglement is not a flaw in a particular model; it is a consequence of joint optimisation, and it is why the discipline replaces line level testing with distributional and behavioural testing.

The practical consequence is the intellectual core of the field: you move from *proving correctness once* to *observing behaviour continuously*. That single shift is what makes MLOps an engineering discipline rather than a toolchain.

3 The technical debt taxonomy

This is the most citable technical content in the field, and quoting it makes the first lecture feel like a research seminar rather than a tools tour.

Debt type	What it means technically
Boundary erosion	Model outputs become inputs to other systems, so the encapsulation you designed no longer holds
Entanglement	The CACE property: no feature is independent
Correction cascades	A model built on the output of another model, patched by a third. Errors compound down the chain
Undeclared consumers	Other teams read your predictions off a table you did not know they had found. Your model becomes a load bearing wall you cannot move
Unstable data dependencies	An upstream signal owned by another team that changes silently. Your accuracy moves and your code does not
Underutilised data dependencies	Features contributing nothing that still create a failure surface
Feedback loops	Your predictions influence the world that generates your next training data
Pipeline jungles and glue code	The scaffolding around the model, typically an order of magnitude larger than the modelling code
Configuration debt	Every threshold, path and flag is a place where a wrong value is silently valid
Changes in the external world	The only dependency that changes without any human touching the system

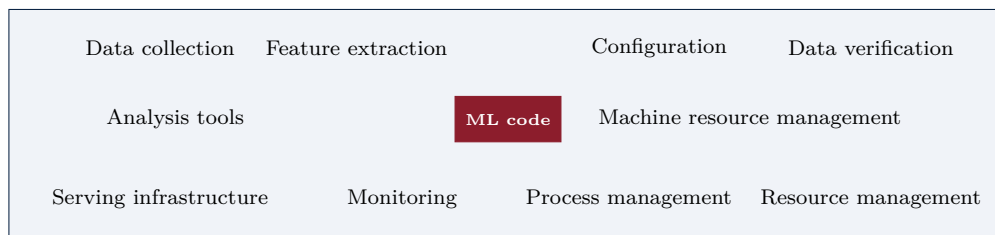


Figure 3: The proportion students never expect. In a real production system the modelling code is a small fraction of the total; the rest is the surrounding engineering, which is exactly the material of this course.

Say the last row of the table slowly in class. It is the sentence that separates this course from every other course they have taken.

4 The anatomy of a production system

There are four subsystems, and confusing them is the most common conceptual error students make.

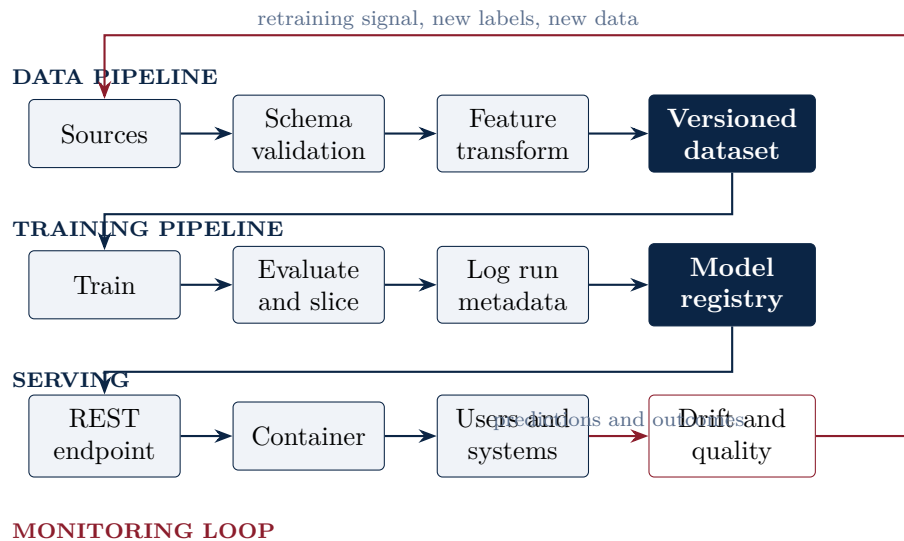


Figure 4: The four subsystems and the one arrow that matters. Without the red return path the system is a line, not a loop, and everything downstream decays silently.

The data pipeline

Ingestion, validation against a schema, transformation and materialisation. The critical property is that transformations applied at training time and at serving time must be identical. When they are not you get training and serving skew, which is invisible offline and catastrophic online.

The training pipeline

A deterministic, parameterised, re runnable job that consumes a versioned dataset and produces a versioned artefact plus metadata. The output is not just weights. It is weights, plus the input signature, plus the environment specification, plus the metrics, plus a lineage pointer to the exact data version.

The serving system

The model behind a network boundary, with a request contract, validation, a latency budget and a concurrency model. Batch scoring and online scoring have entirely different engineering constraints, and students should be able to argue which one a problem needs.

The monitoring loop

Observation of inputs, outputs and eventually outcomes, feeding back into the data pipeline.

5 Training and serving skew, drawn

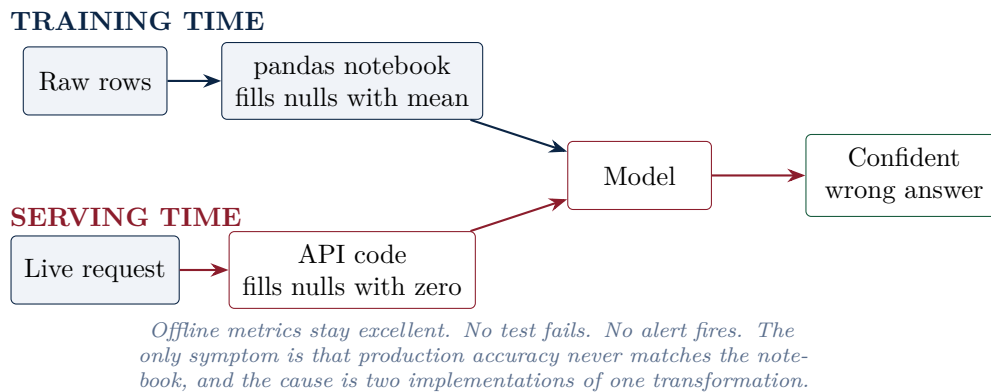


Figure 5: Training and serving skew: the single most common production failure in machine learning, and the reason feature stores and shared transformation libraries exist.

6 Lineage, formally

Every prediction in a mature system can be traced backwards through a directed acyclic graph. If a student can reconstruct this chain for their own project, they have understood the course.

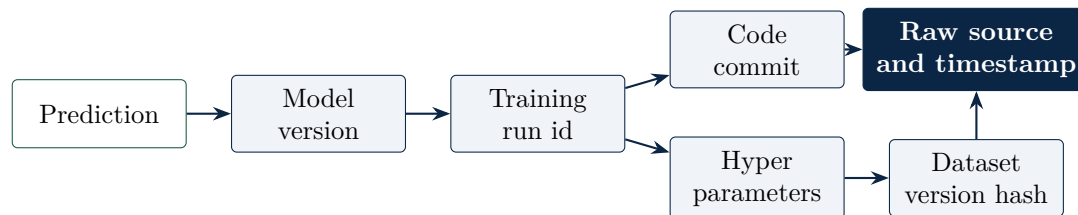


Figure 6: The lineage chain. In an audit, an incident review or any regulated deployment, this graph is the deliverable. Tooling exists to record it automatically, which is the entire argument for a tracking server and a registry.

7 Reproducibility, and its limits

Students think reproducibility means running it twice. There are two distinct standards, and the distinction is worth teaching explicitly.

Standard	What it requires
Statistical reproducibility	The same pipeline gives metrics within a tolerance. Achievable, and usually what is actually required
Bitwise reproducibility	The artefact is byte identical. Much harder, because non determinism enters from seeds, thread counts and scheduling, floating point summation not being associative so parallel reductions vary by order, library and driver versions, and hardware

Teaching this stops the naive belief that setting a seed guarantees determinism, and it gives students precise language in an interview.

8 The three shifts, decomposed

Most courses say drift. A student who can decompose it will stand out. Let the input be x and the label y .

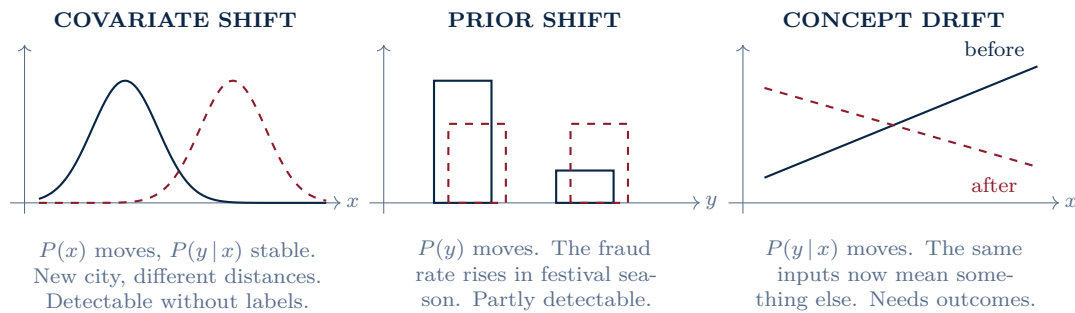


Figure 7: Solid navy is the reference window, dashed red the current window. The engineering consequence is the asymmetry: input drift is cheap to monitor because it needs no ground truth, while concept drift is expensive because it needs labels that arrive late or never.

Detection methods worth naming

Kolmogorov Smirnov for continuous features, chi squared for categorical, Population Stability Index as the industry threshold metric, Jensen Shannon distance and Wasserstein distance for distributional distance, and a domain classifier where you train a model to separate reference from current data and read its AUC as a drift score.

The trap to teach: run a statistical test across two hundred features at the five percent level and roughly ten features will appear to drift every single day by chance alone. Multiple comparison correction, or a severity threshold, is not optional.

9 The quiet failure

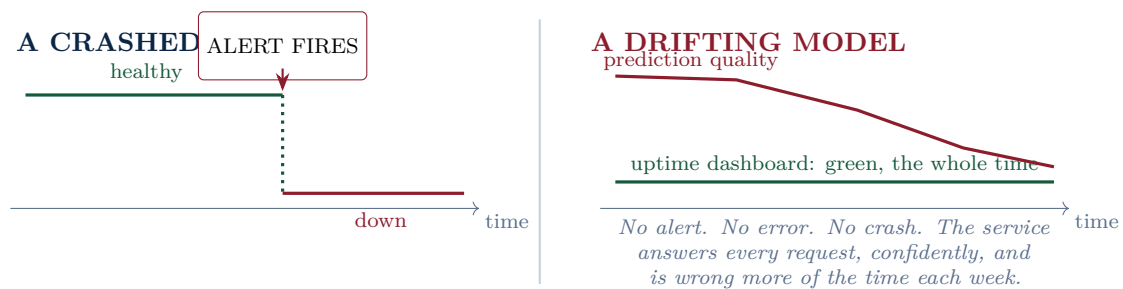


Figure 8: The asymmetry that justifies the entire monitoring module. A web server tells you when it crashes. A model does not tell you when it starts being wrong.

10 Deployment strategies and rollback semantics

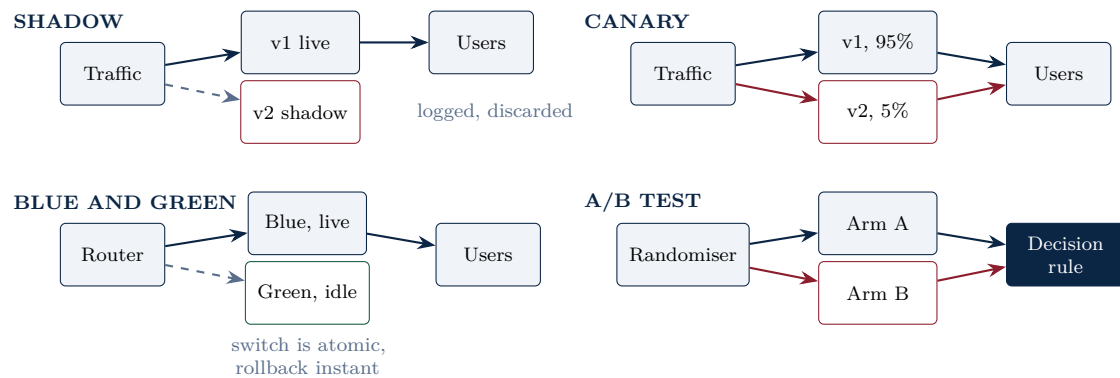


Figure 9: Four ways to put a new model in front of users, ordered by risk appetite. Shadow buys certainty at the cost of compute; canary buys early signal under real load; blue and green buys instant rollback at the cost of double infrastructure; A and B testing buys causal evidence at the cost of time.

Rollback in machine learning is not rollback in software

Redeploying the previous container is not enough. You must restore the previous **model version and the feature transformations that matched it**. A new container with an old model, or an old model with new transformations, reproduces the skew failure from Section 5. This is precisely why the model registry and the dataset version exist.

11 The testing pyramid for machine learning

Software has unit, integration and end to end tests. Machine learning adds three layers students have never met.

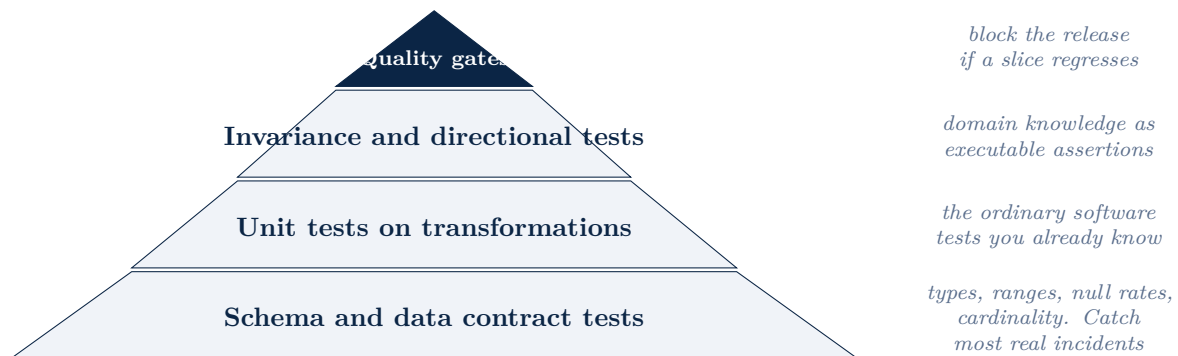


Figure 10: Wide at the base, because most production incidents are data incidents rather than model incidents. Invariance tests are the machine learning equivalent of unit tests: if the customer name changes the prediction must not; if distance increases the predicted time must not decrease.

12 The maturity model

Google's formulation is the reference every practitioner quotes. Ask students to place a company they admire on this scale. The honest answer is usually level zero, which is exactly why the skill is valuable.

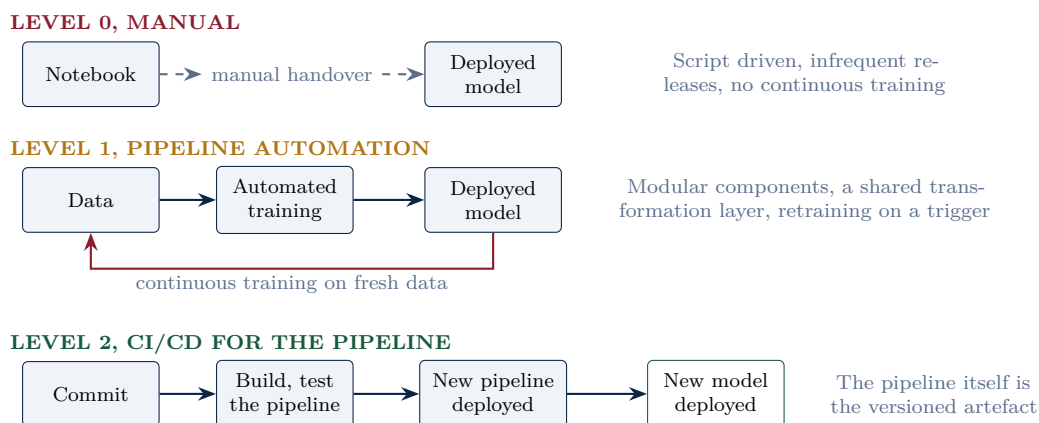


Figure 11: The three levels of automation. Note what changes at level two: the deployed artefact stops being a model and becomes a pipeline that produces models.

PART II

Why It Must Be Taught

13 The curriculum gap, stated without insult

Every prior course optimises an offline metric on a fixed dataset. That is correct pedagogy for learning algorithms and a distorted picture of the work. In industry the objective is different: deliver a decision, within a latency and cost budget, that stays correct while the world moves, and that somebody can audit later.

Nothing in a standard curriculum teaches that objective. Students graduate able to raise accuracy by two points and unable to say why their model cannot be called by another program. The gap is exposure, not intelligence.

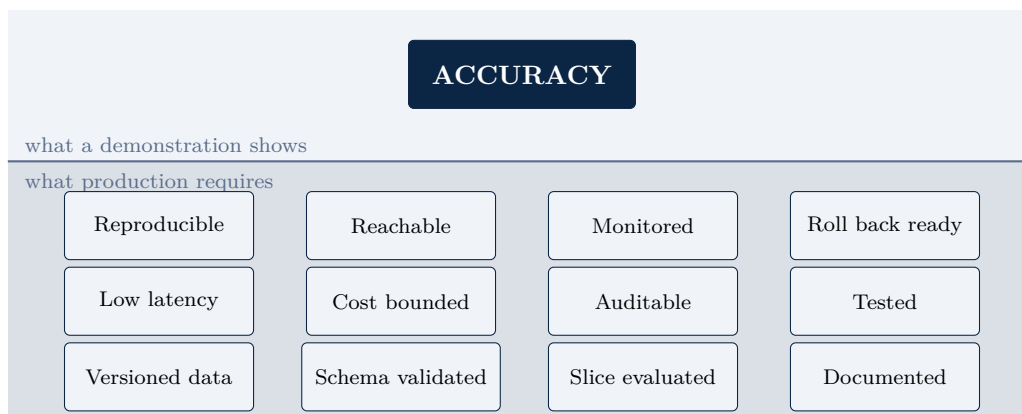


Figure 12: Accuracy is the tip. Everything that keeps a model trustworthy for strangers sits below the waterline, invisible until it is missing, and every box below the line is a session in this syllabus.

14 The economic argument, in one line

A model that is offline costs you its uptime. A model that is silently wrong costs you every decision it makes until somebody notices, and nobody is watching by default. That asymmetry is why observability is a first class engineering concern, and why companies pay for people who build it.

15 The role map

Role	What the job actually is	What the hiring loop tests
Machine learning engineer	Owns the model in production: training pipeline, serving, evaluation	Python and system design, model debugging, some algorithms
MLOps or platform engineer	Builds the infrastructure other teams train and deploy on	Containers, orchestration, CI/CD, cloud, reliability
Data engineer	Owns pipelines, contracts and quality upstream of every model	SQL, distributed processing, orchestration, data modelling

Role	What the job actually is	What the hiring loop tests
Applied scientist	Owens modelling with production responsibility	Modelling depth plus everything in row one
Reliability engineer for ML	Owens availability, latency and incident response for model services	Systems, observability, on call practice

The point to make out loud

Rows two, three and five are less crowded than row one, are reachable from an undergraduate degree, and are almost never taught. A student who finishes this course with a containerised, tested, monitored service has evidence for all three.

16 What interviews actually ask, once they move past data structures

- Design a fraud detection service for a bank. Where does the model sit, what is the latency budget, how do you compute features that need historical aggregation, what do you monitor, and how do you roll back.
- Our model was accurate in January and poor by June. No code changed. Give two explanations and say which stage should have caught each.
- Here is a training script. Review it. The expected findings: no seed, no versioned data, no isolation of the test set, no logging of parameters, hardcoded paths, and a transformation that will not exist at serving time.
- Why is accuracy in your notebook higher than accuracy in production? The word wanted is skew, and the follow up is leakage.
- Walk me through everything that happens between a commit and a deployed model in your project.

Every one of these is a session in the syllabus. Saying that in the first lecture reframes the course from a requirement into an advantage.

17 The portfolio asymmetry

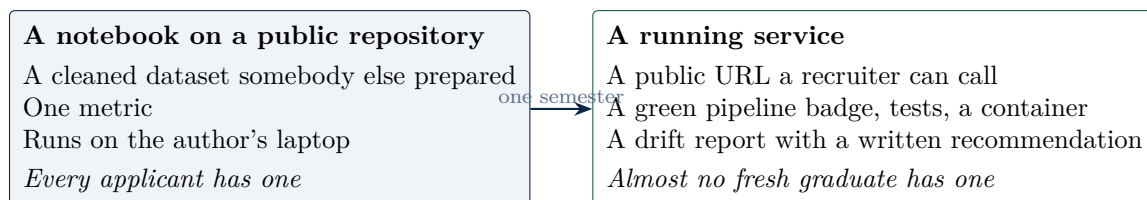


Figure 13: The single most persuasive slide you can show a fifth semester student. The right hand box converts an interview from a quiz into a conversation about their own work.

PART III

The Skill Stack

18 What they must already own, and what they usually lack

Prerequisite	Level needed	Reality in a typical section
Python beyond scripting: modules, imports, packaging	Comfortable	Most can script, few can structure a package
Object oriented design	Working	Taught, rarely used
Command line: paths, environment variables, processes, ports	Working	The largest genuine gap
Git beyond commit: branches, merges, pull requests	Working	Usually only commit and push
HTTP and REST: methods, status codes, JSON	Conceptual	Rarely taught explicitly
Machine learning basics: split, leakage, metrics	Working	Present, with leakage poorly understood
SQL and pandas	Basic	Uneven
Networking basics: localhost, ports, container networks	Conceptual	Almost absent

Plan for the shell, HTTP and packaging gaps explicitly. They are the three that quietly sink the middle of the semester, and each closes in one focused session.

19 The stack we build, layer by layer

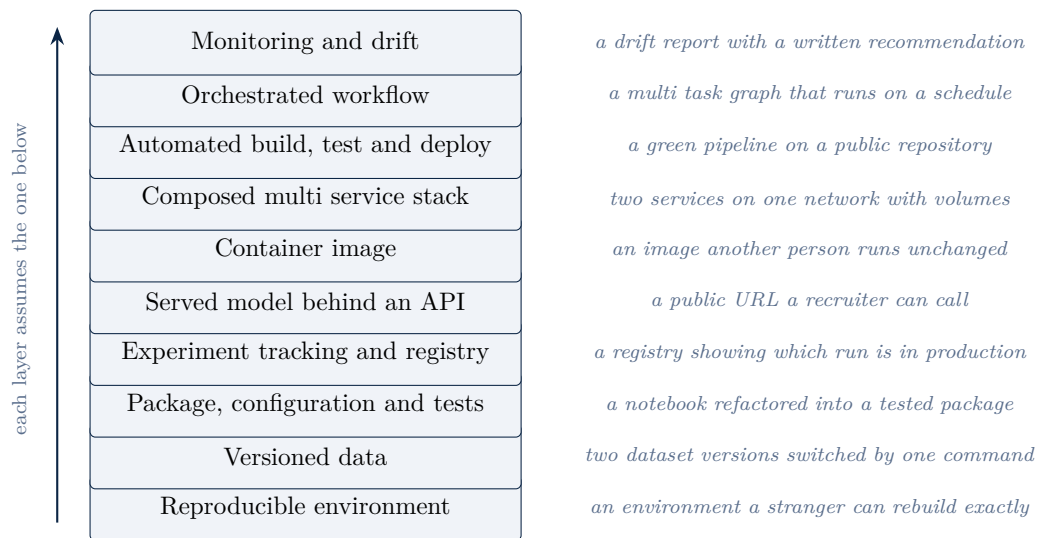


Figure 14: The semester as a ladder. Nothing is thrown away, and every rung produces an artefact a student can show to somebody who is deciding whether to hire them.

20 What compounds beyond this course

Cloud fundamentals and one provider in depth. Infrastructure as code. Distributed data processing. Feature stores. Vector databases and retrieval systems. Evaluation harnesses for generative systems, where prompts become versioned artefacts, evaluation sets become test suites, and cost and latency budgets become first class constraints. Every one of these is easier for somebody who already thinks in artefacts, pipelines and observability.

PART IV

Opening the Course

21 The cold open, twelve minutes

Do not begin with the syllabus. Begin with a failure they can feel.

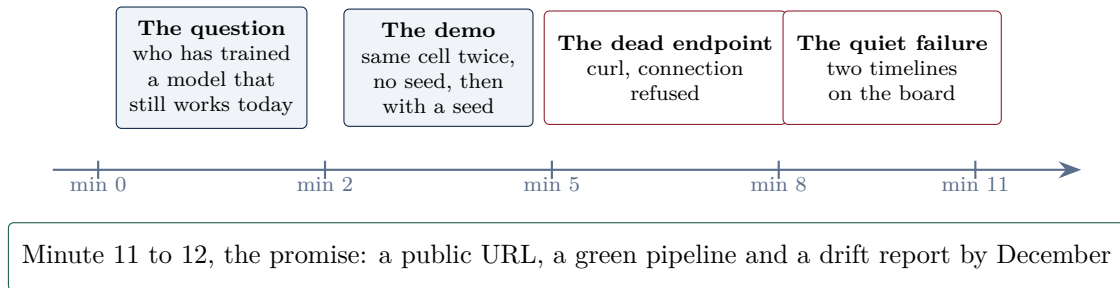


Figure 15: The opening sequence. Three physical demonstrations before a single slide of syllabus. Each takes under three minutes and each lands an idea the rest of the semester rests on.

Minute 0 to 2, the question

"How many of you have trained a model that worked?" Most hands. "How many have given that model to somebody else, and it still worked a month later, without you touching it?" Almost none. "That distance is this course."

Minute 2 to 5, the demonstration

Run the same training cell twice with the seed removed and show two different results. Then set the seed and show identical output and an identical checksum. Reproducibility becomes physical rather than theoretical in three minutes.

Minute 5 to 8, the dead endpoint

Show a request against a model service that is not running, and the connection refused error. Ask: your model has ninety five percent accuracy, and this is what the customer sees. What is the accuracy of a service nobody can reach?

Minute 8 to 11, the quiet failure

Draw the two timelines from Section 10. Then say the line: a web server tells you when it crashes, a model does not tell you when it starts being wrong. It keeps answering, confidently.

Minute 11 to 12, the promise

By December every one of you will have a public URL that a stranger can call, backed by a repository with a green pipeline and a drift report. Roughly nobody graduating this year has that.

22 The four lines for the board

1. Code, data, model. Version all three.
2. Accuracy is the start, not the finish.
3. The world changes even when your code does not.

4. A model nobody can call is a model nobody has.

23 Hard questions from sharp students

Is this not just DevOps with new tools?

No. DevOps assumes the artefact is a deterministic function of the source. Here the artefact is compiled from data, so you inherit three problems DevOps has no answer for: the artefact cannot be diffed meaningfully, tests cannot assert exact outputs, and the system degrades without anybody changing anything. Continuous training has no DevOps counterpart at all.

Why not use a managed platform and skip this?

Managed platforms implement these concepts, they do not remove them. You still choose a deployment strategy, define drift thresholds, design a rollback path and own a latency budget. A platform without the concepts is an expensive way to make the same mistakes faster, and the concepts transfer across providers while the buttons do not.

Does any of this matter for large language models?

More, not less. The artefact set grows to include prompts, retrieval indexes, tool definitions and guardrails, all of which need versioning. Evaluation moves from a single metric to a graded evaluation suite. Cost and latency become primary design constraints. Monitoring adds output quality and refusal rates alongside input drift. The vocabulary in this handbook is exactly the vocabulary that field uses.

Will this be automated away?

Parts of the mechanics, quickly. The judgement will not be: deciding what to monitor, choosing a rollback threshold, defining what correct means for a slice of users, and knowing which failure is a data problem rather than a model problem. Learn the judgement; let the tools churn.

24 The sentence to end the first lecture on

Everything you have been taught so far assumes the world holds still while you finish your work.

It does not.

This course is about what you build so that your work survives the world moving.

PART V

Reference

25 Vocabulary to use fluently from day one

Term	Precise meaning
Training and serving skew	Transformations differ between training and inference, so the model sees a distribution it never trained on
Data leakage	Information unavailable at prediction time contaminates training, inflating offline metrics
Feature store	A shared, versioned store of computed features used by both training and serving to eliminate skew
Model registry	The system of record for model versions and their lifecycle stage
Model signature	The declared input and output schema of a model artefact
Lineage	The traceable chain from a prediction back to raw data and commit
Continuous training	Automated retraining triggered by schedule, data volume or a drift signal
Reference and current window	The two datasets compared to compute drift
Population Stability Index	A distributional distance widely used as an industry drift threshold
Slice based evaluation	Metrics computed on named subgroups rather than the aggregate
Shadow deployment	The new model scores live traffic with its output logged and discarded
Canary release	A fraction of traffic routed to the new version
Champion and challenger	Continuous structured competition between the incumbent and alternatives
Latency budget	The time allowed for a response, usually specified at a high percentile
p95 and p99 latency	Tail latency, which is what users actually experience
Idempotency	Repeating an operation produces the same state, essential for retries in pipelines
Backfill	Running a pipeline over historical intervals
Directed acyclic graph	The dependency structure of a pipeline
Data contract	An enforced agreement about schema and semantics between producer and consumer
Quality gate	An automated threshold that blocks a release
Artefact	Any versioned output: model, dataset, image, report

Term	Precise meaning
Immutable infrastructure	Replace rather than modify, which makes rollback trivial
Observability	The ability to infer internal state from outputs, beyond simple monitoring
CACE	Changing anything changes everything, the entanglement property of learned systems
Correction cascade	Layered patches on upstream model errors, compounding downstream

26 Primary sources

Source	Where
Sculley et al., Hidden Technical Debt in Machine Learning Systems, NeurIPS 2015	https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf
Sambasivan et al., Data Cascades in High Stakes AI, CHI 2021, open access	https://dl.acm.org/doi/10.1145/3411764.3445518
Google Cloud, MLOps maturity model	https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning
Databricks, The Big Book of MLOps	https://www.databricks.com/wp-content/uploads/2022/09/The-Big-Book-of-MLOps-v6-082322.pdf
Stanford CS 329S, Machine Learning Systems Design	https://stanford-cs329s.github.io
Evidently documentation, drift detection	https://docs.evidentlyai.com/quickstart_ml
MLflow documentation	https://mlflow.org/docs/latest/ml/getting-started/

Read Part I sections 2, 3 and 9 before the first lecture. Those three carry the technical authority; the rest of this handbook is scaffolding around them.